

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I E.Ragul /192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Ragul

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

ANSWERS:

1. Reverse Engineering a Recommendation System

System Design:

- **Data Collection:** Collect user clicks, searches, purchases, ratings, and product details.
- **PySpark RDD Processing:** Store user and product data in RDDs.
- **RDD Operations:**
 - `map()` – extract user/product features.
 - `filter()` – remove invalid data.
 - `groupByKey()` – group products based on users.
 - `reduceByKey()` – calculate product popularity or ratings.
- **AI Model:** Predict products that a user is likely to purchase.
- **Intelligent Agent:** Monitors user activity and dynamically updates recommendations.
- **Output:** Personalized product recommendations are displayed to users.

Flow:

User Data → RDD → Feature Processing → Recommendation Model → Intelligent Agent → Updated Recommendations

2. Reverse Engineering a Smart Traffic Management System

System Design:

Data Collection: Sensors and cameras collect vehicle count, speed, and congestion data.

- **RDD Streaming:** Real-time traffic data is converted into RDDs for processing.
- **RDD Operations:**
 - map() – extract traffic information.
 - filter() – remove incorrect sensor data.
 - reduceByKey() – calculate vehicle density.
- **AI Decision:** Analyze congestion and predict traffic conditions.
- **Intelligent Agents:** Traffic agents monitor different roads and coordinate with signal controllers.
- **Decision:** Signals automatically change timing according to traffic density.

- **Flow:**

Sensors/Cameras → RDD Stream → Traffic Analysis → AI Decision → Traffic Agent → Signal Control

3. Reverse Engineering a Fraud Detection Pipeline

System Design:

- **Data Collection:** Collect transaction amount, location, time, device, and information.
- **RDD Processing:** Transaction records are stored and processed using PySpark RDDs.
- **RDD Operations:**
 - map() – extract transaction features.
 - filter() – remove invalid transactions.
 - groupByKey() – group transactions by customer.
 - reduceByKey() – calculate transaction statistics.
- **Feature Extraction:** Identify unusual amounts, locations, and transaction frequency.
- **AI Model:** Detect abnormal transaction patterns.
- **Intelligent Agent:** Generates an alert when fraud probability is high.
- **Output:** Transaction is approved, blocked, or sent for manual verification.

Flow:

Transaction → RDD → Feature Extraction → Anomaly Detection → Fraud Agent → Alert/Block

4. Reverse Engineering a Personalized Learning System

System Design:

- **Data Collection:** Collect marks, quiz results, learning time, mistakes, and student activity.
- **RDD Processing:** Student performance data is stored in RDDs.
- **RDD Operations:**
 - `map()` – calculate student performance.
 - `filter()` – identify weak topics.
 - `groupByKey()` – group activities by student.
 - `reduceByKey()` – calculate average scores.
- **AI Model:** Predict the student's learning level.
- **Intelligent Agent:** Selects suitable content based on student performance.
- **Feedback:** Provides recommendations, easier/harder questions, and corrective feedback.

Flow:

Student Activity → RDD → Performance Analysis → AI Model → Learning Agent → Personalized Content

5. Reverse Engineering a Cloud-Based AI Monitoring System

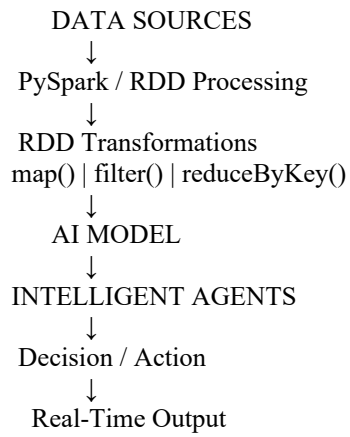
System Design:

- **Data Collection:** Collect CPU usage, memory usage, network traffic, server logs, and error messages.
- **PySpark RDD Processing:** Large-scale logs are distributed across multiple nodes
- **RDD Operations:**
 - `map()` – extract monitoring information.
 - `filter()` – identify errors and abnormal values.
 - `reduceByKey()` – calculate resource usage.
- **Fault Tolerance:** RDDs can recompute lost partitions, allowing processing to continue when a worker fails
- **AI Model:** Detect abnormal resource usage and predict failures.
- **Intelligent Agents:**
 - **Monitoring Agent** – observes system status.
 - **Scaling Agent** – increases/decreases resources.
 - **Anomaly Agent** – detects unusual behavior.
 - **Response Agent** – takes corrective action.
- **Output:** Automatically scales resources and responds to detected failures.

Flow:

Cloud Logs → PySpark RDD → Log Processing → Anomaly Detection → Intelligent Agents →
Scaling/Response

Common Architecture



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation